```
    TABLE 1.  GLOSSARY OF WORDS IN CAMEL80.AZM
    Words which are (usually) written in CODE.

NAME   stack in -- stack out          description

  Guide to stack diagrams:  R: = return stack,
  c = 8-bit character, flag = boolean (0 or -1),
  n = signed 16-bit, u = unsigned 16-bit,
  d = signed 32-bit, ud = unsigned 32-bit,
  +n = unsigned 15-bit, x = any cell value,
  i*x j*x = any number of cell values,
  a-addr = aligned adrs, c-addr = character adrs
  p-addr = I/O port adrs, sys = system-specific.
  Refer to ANS Forth document for more details.

               ANS Forth Core words
These are required words whose definitions are
specified by the ANS Forth document.

!      x a-addr --           store cell in memory
+      n1/u1 n2/u2 -- n3/u3             add n1+n2
+!     n/u a-addr --           add cell to memory
-      n1/u1 n2/u2 -- n3/u3        subtract n1-n2
<      n1 n2 -- flag           test n1<n2, signed
=      x1 x2 -- flag                  test x1=x2
>      n1 n2 -- flag           test n1>n2, signed
>R     x --   R: -- x        push to return stack
?DUP   x -- 0 | x x                DUP if nonzero
@      a-addr -- x         fetch cell from memory
0<     n -- flag             true if TOS negative
0=     n/u -- flag           return true if TOS=0
1+     n1/u1 -- n2/u2                add 1 to TOS
1-     n1/u1 -- n2/u2         subtract 1 from TOS
2*     x1 -- x2             arithmetic left shift
2/     x1 -- x2            arithmetic right shift
AND    x1 x2 -- x3                    logical AND
CONSTANT   n --           define a Forth constant
C!     c c-addr --           store char in memory
C@     c-addr -- c         fetch char from memory
DROP   x --                     drop top of stack
DUP    x -- x x            duplicate top of stack
EMIT   c --           output character to console
EXECUTE   i*x xt -- j*x   execute Forth word 'xt'
EXIT   --                 exit a colon definition
FILL   c-addr u c --        fill memory with char
I      -- n   R: sys1 sys2 -- sys1 sys2
                     get the innermost loop index
INVERT x1 -- x2                 bitwise inversion
J      -- n   R: 4*sys -- 4*sys
                        get the second loop index
KEY    -- c           get character from keyboard
LSHIFT x1 u -- x2        logical L shift u places
NEGATE x1 -- x2                  two's complement
OR     x1 x2 -- x3                     logical OR
OVER   x1 x2 -- x1 x2 x1        per stack diagram
ROT    x1 x2 x3 -- x2 x3 x1     per stack diagram
RSHIFT x1 u -- x2        logical R shift u places
R>     -- x    R: x --      pop from return stack
R@     -- x    R: x -- x       fetch from rtn stk
SWAP   x1 x2 -- x2 x1          swap top two items
UM*    u1 u2 -- ud       unsigned 16x16->32 mult.
UM/MOD ud u1 -- u2 u3     unsigned 32/16->16 div.
UNLOOP --   R: sys1 sys2 --       drop loop parms
U<     u1 u2 -- flag         test u1<n2, unsigned
VARIABLE   --             define a Forth variable
XOR    x1 x2 -- x3                    logical XOR

               ANS Forth Extensions
These are optional words whose definitions are
```

specified by the ANS Forth document.

```
<>     x1 x2 -- flag              test not equal
BYE    i*x --                     return to CP/M
CMOVE  c-addr1 c-addr2 u --      move from bottom
CMOVE> c-addr1 c-addr2 u --        move from top
KEY?   -- flag        return true if char waiting
M+     d1 n -- d2            add single to double
NIP    x1 x2 -- x2              per stack diagram
TUCK   x1 x2 -- x2 x1 x2        per stack diagram
U>     u1 u2 -- flag         test u1>u2, unsigned
```

Private Extensions

These are words which are unique to CamelForth. Many of these are necessary to implement ANS Forth words, but are not specified by the ANS document. Others are functions I find useful.

```
(do)   n1|u1 n2|u2 --  R: -- sys1 sys2
                           run-time code for DO
(loop) R: sys1 sys2 --  | sys1 sys2
                            run-time code for LOOP
(+loop)  n --   R: sys1 sys2 --  | sys1 sys2
                          run-time code for +LOOP
><     x1 -- x2                       swap bytes
?branch  x --                 branch if TOS zero
BDOS   DE C -- A                  call CP/M BDOS
branch --                          branch always
lit    -- x         fetch inline literal to stack
PC!    c p-addr --            output char to port
PC@    p-addr -- c           input char from port
RP!    a-addr --         set return stack pointer
RP@    -- a-addr         get return stack pointer
SCAN   c-addr1 u1 c -- c-addr2 u2
                             find matching char
SKIP   c-addr1 u1 c -- c-addr2 u2
                           skip matching chars
SP!    a-addr --           set data stack pointer
SP@    -- a-addr           get data stack pointer
S=     c-addr1 c-addr2 u -- n      string compare
               n<0: s1<s2, n=0: s1=s2, n>0: s1>s2
USER   n --              define user variable 'n'
```